

Python for Data Science Project

Angel Dalila Helen Siwele

NS.2022.R4G0J9

Submitted on 07/08/2023

This is a report detailing the methods and processes used to generate various visualisations (tables, graphs and plots) for the model/toy data sets, as well as the conclusions that can be drawn from these tables, graphs and diagrams that have been generated .

Source code

Main Class (main.py) Source Code

```
import Question1
import Question2

obj = Question2
generate_data = Question1

menu = "===== " + "\n"
menu = menu + "|          Exam Data          |" + "\n"
menu = menu + "===== " + "\n"
menu = menu + "| 1.Generate Toy Data for Math114 Exam          |" + "\n"
menu = menu + "| 2.Generate Toy Data for CS114 Exam          |" + "\n"
menu = menu + "===== " + "\n"
menu = menu + "| 3.Generate Statistical Visualisations for Math114 Exam Data|" + "\n"
menu = menu + "| 4.Generate Statistical Visualisations for CS114 Exam Data |" + "\n"
menu = menu + "===== " + "\n"
menu = menu + "| X.Exit          |" + "\n"
menu = menu + "===== " + "\n"
menu = menu + "Choice:_"

math_menu = "===== " + "\n"
math_menu = math_menu + "|          Visualisations for Math 114 Exam Data          |" + "\n"
math_menu = math_menu + "===== " + "\n"
math_menu = math_menu + "| 1. Generate Frequency Tables for Math 114 Exam Data |" + "\n"
math_menu = math_menu + "| 2. Generate Bar Graphs for Math 114 Exam Data |" + "\n"
math_menu = math_menu + "| 3. Generate Line Graphs for Math 114 Exam Data |" + "\n"
math_menu = math_menu + "| 4. Generate Scatter Plots for Math 114 Exam Data |" + "\n"
math_menu = math_menu + "===== " + "\n"
math_menu = math_menu + "| B. Back          |" + "\n"
math_menu = math_menu + "| X. Exit          |" + "\n"
math_menu = math_menu + "===== " + "\n"
math_menu = math_menu + "Choice:_"

cs_menu = "===== " + "\n"
cs_menu = cs_menu + "| Visualisations for Computer Science 114 Exam Data |" + "\n"
cs_menu = cs_menu + "===== " + "\n"
cs_menu = cs_menu + "| 1. Generate Frequency Tables for CS 114 Exam Data |" + "\n"
cs_menu = cs_menu + "| 2. Generate Bar Graphs for CS 114 Exam Data |" + "\n"
cs_menu = cs_menu + "| 3. Generate Line Graphs for CS 114 Exam Data |" + "\n"
cs_menu = cs_menu + "| 4. Generate Scatter Plots for CS 114 Exam Data |" + "\n"
cs_menu = cs_menu + "===== " + "\n"
cs_menu = cs_menu + "| B. Back          |" + "\n"
cs_menu = cs_menu + "| X. Exit          |" + "\n"
cs_menu = cs_menu + "===== " + "\n"
cs_menu = cs_menu + "Choice:_"

def main():
    sub_menu = ""
    answer = input(menu)
    if int(answer) == 1:
        generate_data.generate_math_exam_data()
    elif int(answer) == 2:
        generate_data.generate_cs_exam_data()
```

```

elif int(answer) == 3:
    sub_menu = input(math_menu)
    # all math114 visualisations
    if answer == "A" or answer == "a":
        obj.freq_student_marks_math114()
        obj.freq_student_ages_math114()
        obj.freq_avg_study_hrs_math114()
        obj.bar_age_group_math114()
        obj.line_marks_study_hrs_math114()
        obj.scatter_marks_time_math114()
        obj.scatter_study_time_age_math114()
    # math114 freq tables
    elif int(sub_menu) == 1:
        obj.freq_student_marks_math114()
        obj.freq_student_ages_math114()
        obj.freq_avg_study_hrs_math114()
    # math114 bar graphs
    elif int(sub_menu) == 2:
        obj.bar_age_group_math114()
    # math114 line graphs
    elif int(sub_menu) == 3:
        obj.line_marks_study_hrs_math114()
    # math114 scatter plots
    elif int(sub_menu) == 4:
        obj.scatter_marks_time_math114()
        obj.scatter_study_time_age_math114()
    # terminate program
    elif answer == "X" or answer == "x":
        exit()
    else:
        main()
elif int(answer) == 4:
    sub_menu = input(cs_menu)
    # all math114 visualisations
    if answer == "A" or answer == "a":
        obj.freq_student_marks_cs114()
        obj.freq_student_ages_cs114()
        obj.freq_avg_study_hrs_cs114()
        obj.bar_age_group_cs114()
        obj.line_marks_study_hrs_cs114()
        obj.scatter_marks_time_cs114()
        obj.scatter_study_time_age_cs114()
    # cs114 freq tables
    elif int(sub_menu) == 1:
        obj.freq_student_marks_cs114()
        obj.freq_student_ages_cs114()
        obj.freq_avg_study_hrs_cs114()
    # cs114 bar graphs
    elif int(sub_menu) == 2:
        obj.bar_age_group_cs114()
    # cs114 line graphs
    elif int(sub_menu) == 3:
        obj.line_marks_study_hrs_cs114()
    # cs114 scatter plots
    elif int(sub_menu) == 4:
        obj.scatter_marks_time_cs114()
        obj.scatter_study_time_age_cs114()
    # back to main menu
    elif answer == "B" or answer == "b":
        main()
    # terminate program
    elif answer == "X" or answer == "x":

```

```
        exit()
    else:
        main()
if answer == "X" or answer == "x":
    exit()
else:
    main()
main()
```

Question 1 Source Code used to Generate Model CSV Files

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import csv
import random

# QUESTION 1 Create two datasets in csv files, one for each examination, with 150 rows each.
# Writing data to CSV exam files
csv_headers = ['Student Number', 'Age', 'Average Hours Studying On Campus', 'Student Mark',
'Time Taken on Exam']

def generate_student_data():
    student_number = random.randint(1000, 9999)
    age_group = random.randint(18, 60)
    hours_studying = random.uniform(1, 5)
    student_mark = random.randint(0, 130)
    time_taken_on_exam = random.randint(0, 180)
    return [student_number, age_group, hours_studying, student_mark, time_taken_on_exam]

def generate_math_exam_data():
    # Generating 150 rows of data for math114 exam file
    data_one = []
    for _ in range(150):
        data_one.append(generate_student_data())

    with open('student_data_math114.csv', mode='w', newline='') as file:
        writer = csv.writer(file)
        writer.writerow(csv_headers)
        writer.writerows(data_one)
    print("CSV file 'student_data_math114.csv' with 150 rows has been created.")

def generate_cs_exam_data():
    # Generating 150 rows of data for cs114 exam file
    data_two = []
    for _ in range(150):
        data_two.append(generate_student_data())

    with open('student_data_cs114.csv', mode='w', newline='') as file:
        writer = csv.writer(file)
        writer.writerow(csv_headers)
        writer.writerows(data_two)
    print("CSV file 'student_data_cs114.csv' with 150 rows has been created.")
```

Question 2 Full Source Code used to Generate Data Visualisations

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# QUESTION 2 Before performing operations on your data, you need to clean your data, remove any zero values and document the
# process thoroughly in your report.
# 2.1 Create frequency tables for math114 and cs114 exams based on the following headings, and using the following criteria:

# 2.1.1 a) Create frequency table showing average hours Math114 students spent studying on campus
def freq_avg_study_hrs_math114():
    data = {}
    # count number of students in each category of hours using the following counters
    one_two_hours_math114counter = 0
    two_three_hours_math114counter = 0
    three_four_hours_math114counter = 0
    four_five_hours_math114counter = 0
    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if float(y[2]) < 2:
                one_two_hours_math114counter += 1
            elif float(y[2]) < 3:
                two_three_hours_math114counter += 1
            elif float(y[2]) < 4:
                three_four_hours_math114counter += 1
            elif float(y[2]) <= 5:
                four_five_hours_math114counter += 1

    data['Average Hours Studying'] = ['1 - 2', '2 - 3', '3 - 4', '4 - 5']
    data['Frequency'] = [one_two_hours_math114counter, two_three_hours_math114counter, three_four_hours_math114counter,
four_five_hours_math114counter]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "Average Hours Spent on Campus by Math114 Students"
    print(heading)
    print(frequency_table_df.to_string(index=False))

# 2.1.1 b) Create frequency table showing average hours CS114 students spent studying on campus
def freq_avg_study_hrs_cs114():
    data = {}
    # count number of students in each category of hours using the following counters
    one_two_hours_cs114counter = 0
    two_three_hours_cs114counter = 0
    three_four_hours_cs114counter = 0
    four_five_hours_cs114counter = 0
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if float(y[2]) < 2:
                one_two_hours_cs114counter += 1
            elif float(y[2]) < 3:
                two_three_hours_cs114counter += 1
            elif float(y[2]) < 4:
```

```

        three_four_hours_cs114counter += 1
    elif float(y[2]) <= 5:
        four_five_hours_cs114counter += 1

    data['Average Hours Studying'] = ['1 - 2', '2 - 3', '3 - 4', '4 - 5']
    data['Frequency'] = [one_two_hours_cs114counter, two_three_hours_cs114counter, three_four_hours_cs114counter,
four_five_hours_cs114counter]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "Average Hours Spent on Campus by CS114 Students"
    print(heading)
    print(frequency_table_df.to_string(index=False))

```

2.1.2 a) Create frequency table showing student ages of Math114 Students

```

def freq_student_ages_math114():
    data = {}
    eighteen_twenty_five_age_math114counter = 0
    twenty_five_thirty_five_age_math114counter = 0
    thirty_five_forty_five_age_math114counter = 0
    over_forty_five_age_math114counter = 0
    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if int(y[1]) <= 25:
                eighteen_twenty_five_age_math114counter += 1
            elif int(y[1]) <= 35:
                twenty_five_thirty_five_age_math114counter += 1
            elif int(y[1]) <= 45:
                thirty_five_forty_five_age_math114counter += 1
            elif int(y[1]) > 45:
                over_forty_five_age_math114counter += 1

    data['Age Group'] = ['18 - 25', '25 - 35', '35 - 45', 'over 45']
    data['Frequency'] = [eighteen_twenty_five_age_math114counter, twenty_five_thirty_five_age_math114counter,
thirty_five_forty_five_age_math114counter, over_forty_five_age_math114counter]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "Math114 Student Age "
    print(heading)
    print(frequency_table_df.to_string(index=False))

```

2.1.2 b) Create frequency table showing student ages of CS114 Students

```

def freq_student_ages_cs114():
    data = {}
    eighteen_twenty_five_age_cs114counter = 0
    twenty_five_thirty_five_age_cs114counter = 0
    thirty_five_forty_five_age_cs114counter = 0
    over_forty_five_age_cs114counter = 0
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if int(y[1]) <= 25:
                eighteen_twenty_five_age_cs114counter += 1
            elif int(y[1]) <= 35:
                twenty_five_thirty_five_age_cs114counter += 1
            elif int(y[1]) <= 45:
                thirty_five_forty_five_age_cs114counter += 1

```



```

        elif int(y[1]) > 45:
            over_forty_five_age_cs114counter += 1

    data['Age Group'] = ['18 - 25', '25 - 35', '35 - 45', 'over 45']
    data['Frequency'] = [eighteen_twenty_five_age_cs114counter, twenty_five_thirty_five_age_cs114counter,
thirty_five_forty_five_age_cs114counter, over_forty_five_age_cs114counter]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "CS114 Students Age "
    print(heading)
    print(frequency_table_df.to_string(index=False))

# 2.1.3 a) Create frequency table showing Student marks of Math114 Students
def freq_student_marks_math114():
    data = {}
    under_forty_five_math114 = 0
    forty_five_forty_nine_math114 = 0
    fifty_fifty_nine_math114 = 0
    sixty_sixty_nine_math114 = 0
    seventy_seventy_nine_math114 = 0
    over_eighty_math114 = 0

    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if ((float(y[3])/130)*100 < 45:
                under_forty_five_math114 += 1
            elif ((float(y[3])/130)*100 < 50:
                forty_five_forty_nine_math114 += 1
            elif ((float(y[3])/130)*100 < 60:
                fifty_fifty_nine_math114 += 1
            elif ((float(y[3])/130)*100 < 70:
                sixty_sixty_nine_math114 += 1
            elif ((float(y[3])/130)*100 < 80:
                seventy_seventy_nine_math114 += 1
            elif ((float(y[3])/130)*100 >= 80:
                over_eighty_math114 += 1

    data['Mark'] = ['under 45', '45 - 49', '50 - 59', '60 - 69', '70 - 79', 'over 80']
    data['Frequency'] = [under_forty_five_math114, forty_five_forty_nine_math114, fifty_fifty_nine_math114,
sixty_sixty_nine_math114, seventy_seventy_nine_math114, over_eighty_math114]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "Math114 Student Marks "
    print(heading)
    print(frequency_table_df.to_string(index=False))

# 2.1.3 b) Create frequency table showing Student marks of CS114 Students
def freq_student_marks_cs114():
    data = {}
    under_forty_five_cs114 = 0
    forty_five_forty_nine_cs114 = 0
    fifty_fifty_nine_cs114 = 0
    sixty_sixty_nine_cs114 = 0
    seventy_seventy_nine_cs114 = 0
    over_eighty_cs114 = 0

    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()

```

```

for i in range(150):
    line = f.readline()
    y = line.split(',')
    if ((float(y[3])/130)*100 < 45:
        under_forty_five_cs114 += 1
    elif ((float(y[3])/130)*100 < 50:
        forty_five_forty_nine_cs114 += 1
    elif ((float(y[3])/130)*100 < 60:
        fifty_fifty_nine_cs114 += 1
    elif ((float(y[3])/130)*100 < 70:
        sixty_sixty_nine_cs114 += 1
    elif ((float(y[3])/130)*100 < 80:
        seventy_seventy_nine_cs114 += 1
    elif ((float(y[3])/130)*100 >= 80:
        over_eighty_cs114 += 1

data['Mark'] = ['under 45','45 - 49','50 - 59','60 - 69','70 - 79','over 80']
data['Frequency'] = [under_forty_five_cs114, forty_five_forty_nine_cs114, fifty_fifty_nine_cs114, sixty_sixty_nine_cs114,
seventy_seventy_nine_cs114, over_eighty_cs114]
# Creating a DataFrame from the frequency table data
frequency_table_df = pd.DataFrame(data)
heading = "CS114 Student Marks "
print(heading)
print(frequency_table_df.to_string(index=False))

```

2.2 Using Matplotlib, complete the following:

2.2.1 a) Use a bar chart to show the age groups (x) and numbers of Math114 students

```

def bar_age_group_math114():
    # count how many student in each age range and put in array
    student_age_num = np.array([])
    # counters for number of students in each age group
    eighteen_twenty_five_age_math114counter = 0
    twenty_five_thirty_five_age_math114counter = 0
    thirty_five_forty_five_age_math114counter = 0
    over_forty_five_age_math114counter = 0
    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            line_info = line.split(',')
            if 18 <= int(line_info[1]) < 25:
                eighteen_twenty_five_age_math114counter += 1
            elif 25 <= int(line_info[1]) < 35:
                twenty_five_thirty_five_age_math114counter += 1
            elif 35 <= int(line_info[1]) < 45:
                thirty_five_forty_five_age_math114counter += 1
            elif int(line_info[1]) > 45:
                over_forty_five_age_math114counter += 1

    student_age_num = np.append(student_age_num, eighteen_twenty_five_age_math114counter)
    student_age_num = np.append(student_age_num, twenty_five_thirty_five_age_math114counter)
    student_age_num = np.append(student_age_num, thirty_five_forty_five_age_math114counter)
    student_age_num = np.append(student_age_num, over_forty_five_age_math114counter)

    x = np.array(['18 - 25', '25 - 35', '35 - 45', 'over 45'])
    y = student_age_num

    # labelling x-axis and y-axis
    plt.xlabel('Age Group')
    plt.ylabel('Number of Students')

```

```
# displaying the title
plt.title("Number of Math114 Students Per Age Group")
plt.bar(x, y)
plt.show()
```

2.2.1 b) Use a bar chart to show the age groups (x) and numbers of CS114 students

```
def bar_age_group_cs114():
    # count how many student in each age range and put in array
    student_age_num = np.array([])
    # counters for number of students in each age group
    eighteen_twenty_five_age_cs114counter = 0
    twenty_five_thirty_five_age_cs114counter = 0
    thirty_five_forty_five_age_cs114counter = 0
    over_forty_five_age_cs114counter = 0
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            line_info = line.split(',')
            if 18 <= int(line_info[1]) < 25:
                eighteen_twenty_five_age_cs114counter += 1
            elif 25 <= int(line_info[1]) < 35:
                twenty_five_thirty_five_age_cs114counter += 1
            elif 35 <= int(line_info[1]) < 45:
                thirty_five_forty_five_age_cs114counter += 1
            elif int(line_info[1]) > 45:
                over_forty_five_age_cs114counter += 1

    student_age_num = np.append(student_age_num, eighteen_twenty_five_age_cs114counter)
    student_age_num = np.append(student_age_num, twenty_five_thirty_five_age_cs114counter)
    student_age_num = np.append(student_age_num, thirty_five_forty_five_age_cs114counter)
    student_age_num = np.append(student_age_num, over_forty_five_age_cs114counter)

    x = np.array(['18 - 25', '25 - 35', '35 - 45', 'over 45'])
    y = student_age_num

    # labelling x-axis and y-axis
    plt.xlabel('Age Group')
    plt.ylabel('Number of Students')
    # displaying the title
    plt.title("Number of CS114 Students Per Age Group")
    plt.bar(x, y)
    plt.show()
```

2.2.2 a) Use a line graph to show if there is a correlation between higher Math114 marks(y) and more time spent on campus studying(x).

variables to store num of student in each time category

```
def line_marks_study_hrs_math114():
    num_students_one_math114 = 0
    num_students_two_math114 = 0
    num_students_three_math114 = 0
    num_students_four_math114 = 0

    sum_grades_one_math114 = 0
    sum_grades_two_math114 = 0
    sum_grades_three_math114 = 0
    sum_grades_four_math114 = 0
```

```
# get average study hours for each group
```

```

with open('student_data_math114.csv', mode='r', newline='') as f:
    f.readline()
    for i in range(150):
        line = f.readline()
        y = line.split(',')
        if float(y[2]) < 2:
            # get percentage of each student's mark, then add it to grades_students_one_two_math114
            sum_grades_one_math114 = sum_grades_one_math114 + (int(y[3]) / 130) * 100
            num_students_one_math114 += 1
        elif float(y[2]) < 3:
            sum_grades_two_math114 = sum_grades_two_math114 + (int(y[3]) / 130) * 100
            num_students_two_math114 += 1
        elif float(y[2]) < 4:
            sum_grades_three_math114 = sum_grades_three_math114 + (int(y[3]) / 130) * 100
            num_students_three_math114 += 1
        elif float(y[2]) <= 5:
            sum_grades_four_math114 = sum_grades_four_math114 + (int(y[3]) / 130) * 100
            num_students_four_math114 += 1

avg_one_math114 = sum_grades_one_math114 / num_students_one_math114
avg_two_math114 = sum_grades_two_math114 / num_students_two_math114
avg_three_math114 = sum_grades_three_math114 / num_students_three_math114
avg_four_math114 = sum_grades_four_math114 / num_students_four_math114

x = np.array(['1 - 2', '2 - 3', '3 - 4', '4 - 5'])
y = np.array([avg_one_math114, avg_two_math114, avg_three_math114, avg_four_math114])

plt.plot(x, y)
# labelling x-axis and y-axis
plt.xlabel('Time Spent on Campus Studying')
plt.ylabel('Student Marks')
# displaying the title
plt.title('Math114 Student Marks and Time Spent on Campus Studying')
plt.show()

```

#2.2.2 b) Use a line graph to show if there is a correlation between higher CS114 marks(y) and more time spent on campus(x).

```

def line_marks_study_hrs_cs114():
    num_students_one_cs114 = 0
    num_students_two_cs114 = 0
    num_students_three_cs114 = 0
    num_students_four_cs114 = 0

    sum_grades_one_cs114 = 0
    sum_grades_two_cs114 = 0
    sum_grades_three_cs114 = 0
    sum_grades_four_cs114 = 0

    # get average study hours for each group
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if float(y[2]) < 2:
                # get percentage of each student's mark, then add it to grades_students_one_two_math114
                sum_grades_one_cs114 = sum_grades_one_cs114 + (float(y[3]) / 130) * 100
                num_students_one_cs114 += 1
            elif float(y[2]) < 3:
                sum_grades_two_cs114 = sum_grades_two_cs114 + (float(y[3]) / 130) * 100
                num_students_two_cs114 += 1
            elif float(y[2]) < 4:

```

```

        sum_grades_three_cs114 = sum_grades_three_cs114 + (float(y[3]) / 130) * 100
        num_students_three_cs114 += 1
    elif float(y[2]) < 5:
        sum_grades_four_cs114 = sum_grades_four_cs114 + (float(y[3]) / 130) * 100
        num_students_four_cs114 += 1

# get percentage of each student's mark, then add it to grades_students_one_two_math114
avg_one_cs114 = sum_grades_one_cs114 / num_students_one_cs114
avg_two_cs114 = sum_grades_two_cs114 / num_students_two_cs114
avg_three_cs114 = sum_grades_three_cs114 / num_students_three_cs114
avg_four_cs114 = sum_grades_four_cs114 / num_students_four_cs114

x = np.array(['1 - 2', '2 - 3', '3 - 4', '4 - 5'])
y = np.array([avg_one_cs114, avg_two_cs114, avg_three_cs114, avg_four_cs114])

plt.plot(x, y)
# labelling x-axis and y-axis
plt.xlabel('Time Spent on Campus Studying')
plt.ylabel('Student Mark')
# displaying the title
plt.title('CS114 Student Marks and Time Spent on Campus Studying')
plt.show()

# 2.2.3 a) Use a scatter chart to plot each student's mark(y) and the time(x) taken on the Math114 examination, in minutes.
"""
Sort csv file lines asc via bubble sort function
populate x and y numpy arrays
plot scatter plot
"""

# sort array by time taken on exam
def sort_array(array):
    # bubble sort
    for first_idx, first_item in enumerate(array):
        for second_idx, second_item in enumerate(array):
            if int(array[first_idx][4]) < int(array[second_idx][4]):
                # TODO: make sort make sense
                smaller = array[second_idx]
                bigger = array[first_idx]
                array[first_idx] = smaller
                array[second_idx] = bigger
    return array

# sort by hours spent on campus studying
def sort_array_by_study_time(array):
    # bubble sort
    for first_idx, first_item in enumerate(array):
        for second_idx, second_item in enumerate(array):
            if float(array[first_idx][2]) < float(array[second_idx][2]):
                # TODO: make sort make sense
                smaller = array[second_idx]
                bigger = array[first_idx]
                array[first_idx] = smaller
                array[second_idx] = bigger
    return array

def scatter_marks_time_math114():
    x = np.array([]) # time taken
    y = np.array([]) # student mark
    new_array = []
    with open('student_data_math114.csv', mode='r', newline='') as file:
        file.readline()
        for line in file:

```

```

        stripped_line = line.rstrip("\r\n")
        line_items = stripped_line.split(',')
        new_array.append(line_items)

# sort array of lines in ascending order by time which is index 4 of each line
sorted_array = sort_array(new_array)
for sorted_item in sorted_array:
    # [student_number, age_group, hours_studying, student_mark, time_taken_on_exam]
    x = np.append(x, int(sorted_item[4]))
    y = np.append(y, (float(sorted_item[3])/130) * 100)

plt.scatter(x, y)
# labelling x-axis and y-axis
plt.xlabel('Time Spent On Exam')
plt.ylabel('Student Mark')
# # displaying the title
plt.title('Student Marks and Time spent on Math114 Exam')
plt.show()

# 2.2.3 b) Use a scatter chart to plot each student's mark(y) and the time(x) taken on the CS114 examination, in minutes.
def scatter_marks_time_cs114():
    x = np.array([]) # time taken
    y = np.array([]) # student mark
    new_array = []
    with open('student_data_cs114.csv', mode='r', newline=") as file:
        file.readline()
        for line in file:
            stripped_line = line.rstrip("\r\n")
            line_items = stripped_line.split(',')
            new_array.append(line_items)

# sort array of lines in ascending order by time which is index 4 of each line
sorted_array = sort_array(new_array)
for sorted_item in sorted_array:
    # [student_number, age_group, hours_studying, student_mark, time_taken_on_exam]
    x = np.append(x, int(sorted_item[4]))
    y = np.append(y, (float(sorted_item[3]) / 130) * 100)

plt.scatter(x, y)
# labelling x-axis and y-axis
plt.xlabel('Time Spent On Exam')
plt.ylabel('Student Mark')
# # displaying the title
plt.title('Student Marks and Time spent on CS114 Exam')
plt.show()

# 2.2.4 a) Use a scatter chart to plot the relationship between time spent on campus(y) and age of Math114 students.
def scatter_study_time_age_math114():
    x = np.array([])
    y = np.array([])
    new_array = []
    with open('student_data_math114.csv', mode='r', newline=") as f:
        f.readline()
        for line in f:
            stripped_line = line.rstrip("\r\n")
            line_items = stripped_line.split(',')
            new_array.append(line_items)

# sort array of lines in ascending order by time which is index 2 of each line
sorted_array = sort_array_by_study_time(new_array)
for sorted_item in sorted_array:

```

```

        # [student_number, age_group, hours_studying, student_mark, time_taken_on_exam]
        x = np.append(x, float(sorted_item[1]))
        y = np.append(y, float(sorted_item[2]))

plt.scatter(x, y)
#labelling x-axis and y-axis
plt.xlabel('Student Age')
plt.ylabel('Time Spent Studying on Campus')
# displaying the title
plt.title('Time Spent on Studying on Campus and Ages of Math114 students')
plt.show()

# 2.2.4 b) Use a scatter chart to plot the relationship between time spent on campus(y)and age of CS114 students.
def scatter_study_time_age_cs114():
    x = np.array([]) # time spent
    y = np.array([])
    new_array = []
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for line in f:
            stripped_line = line.rstrip('\r\n')
            line_items = stripped_line.split(',')
            new_array.append(line_items)

        # sort array of lines in ascending order by time which is index 2 of each line
        sorted_array = sort_array_by_study_time(new_array)
        for sorted_item in sorted_array:
            # [student_number, age_group, hours_studying, student_mark, time_taken_on_exam]
            x = np.append(x, float(sorted_item[1]))
            y = np.append(y, float(sorted_item[2]))

plt.scatter(x, y)
#labeling x-axis and y-axis
plt.xlabel('Student Age')
plt.ylabel('Time Spent Studying on Campus')
# displaying the title
plt.title('Time Spent on Studying on Campus and Ages of CS114 students')
plt.show()

```

Directions on How To Use The Exam Data Visualisation Application (With Screenshots)

For ease of use, this project has been written as an application with three classes. The Question1.py class contains methods to generate model exam data. The Question2.py class contains all the methods to be used to generate various data visualisations from the exam data (in csv files). And the main.py class is the class from which all the functions from the two other classes are called.

Before running the application, ensure Python and a Python Interpreter are installed. Also make sure to install Numpy, Matplotlib and Pandas libraries.

1. To run the application open the main.py file in a Python-friendly IDE and click the play icon or press Shift and F10. The following menu will then appear in the terminal.

```
=====
|                               |
|           Exam Data         |
|                               |
| 1.Generate Toy Data for Math114 Exam |
| 2.Generate Toy Data for CS114 Exam   |
|                               |
| 3.Generate Statistical Visualisations for Math114 Exam Data|
| 4.Generate Statistical Visualisations for CS114 Exam Data |
|                               |
| X.Exit                           |
|                               |
=====
Choice:_
```

2. To create toy/model exam data select 1 or 2.
3. To generate data visualisations for Math114, select 3.
4. The following menu will appear. Select the specific data visualisation you want to generate from student_data_math114.csv data.

Math 114 Data Visualisations:

Math 114 Frequency Tables:

Table:

Average Hours Spent on Campus by Math114 Students	
Average Hours Studying	Frequency
1 - 2	39
2 - 3	40
3 - 4	39
4 - 5	32

Source Code:

```
# 2.1.1 a) Create frequency table showing average hours Math114 students spent studying on campus
def freq_avg_study_hrs_math114():
    data = {}
    # count number of students in each category of hours using the following counters
    one_two_hours_math114counter = 0
    two_three_hours_math114counter = 0
    three_four_hours_math114counter = 0
    four_five_hours_math114counter = 0
    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if float(y[2]) < 2:
                one_two_hours_math114counter += 1
            elif float(y[2]) < 3:
                two_three_hours_math114counter += 1
            elif float(y[2]) < 4:
                three_four_hours_math114counter += 1
            elif float(y[2]) <= 5:
                four_five_hours_math114counter += 1

    data['Average Hours Studying'] = ['1 - 2', '2 - 3', '3 - 4', '4 - 5']
    data['Frequency'] = [one_two_hours_math114counter, two_three_hours_math114counter,
three_four_hours_math114counter, four_five_hours_math114counter]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "Average Hours Spent on Campus by Math114 Students"
    print(heading)
    print(frequency_table_df.to_string(index=False))
```

Conclusion:

The number of students in each group of Average Study Hours is evenly distributed.

Table:

Math114 Student Age	
Age Group	Frequency
18 - 25	22
25 - 35	38
35 - 45	31
over 45	59

Source Code:

```
# 2.1.2 a) Create frequency table showing student ages of Math114 Students
def freq_student_ages_math114():
    data = {}
    eighteen_twenty_five_age_math114counter = 0
    twenty_five_thirty_five_age_math114counter = 0
    thirty_five_forty_five_age_math114counter = 0
    over_forty_five_age_math114counter = 0
    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if int(y[1]) <= 25:
                eighteen_twenty_five_age_math114counter += 1
            elif int(y[1]) <= 35:
                twenty_five_thirty_five_age_math114counter += 1
            elif int(y[1]) <= 45:
                thirty_five_forty_five_age_math114counter += 1
            elif int(y[1]) > 45:
                over_forty_five_age_math114counter += 1

    data['Age Group'] = ['18 - 25', '25 - 35', '35 - 45', 'over 45']
    data['Frequency'] = [eighteen_twenty_five_age_math114counter,
twenty_five_thirty_five_age_math114counter, thirty_five_forty_five_age_math114counter,
over_forty_five_age_math114counter]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "Math114 Student Age "
    print(heading)
    print(frequency_table_df.to_string(index=False))
```

Conclusion:

The age group with the lowest number of Math 114 students is the 18 - 25 group. The age group with the second lowest number of Math 114 students is the 25 - 35 group. The age group with the second highest number of Math 114 students is the 35 - 45. The age group with the highest number of Math 114 students is the over 45. This shows that there are more students over 35, than under 35.

Table:

Math114 Student Marks

Mark	Frequency
under 45	69
45 - 49	9
50 - 59	13
60 - 69	19
70 - 79	16
over 80	24

Source Code:

```
# 2.1.3 a) Create frequency table showing Student marks of Math114 Students
def freq_student_marks_math114():
    data = {}
    under_forty_five_math114 = 0
    forty_five_forty_nine_math114 = 0
    fifty_fifty_nine_math114 = 0
    sixty_sixty_nine_math114 = 0
    seventy_seventy_nine_math114 = 0
    over_eighty_math114 = 0

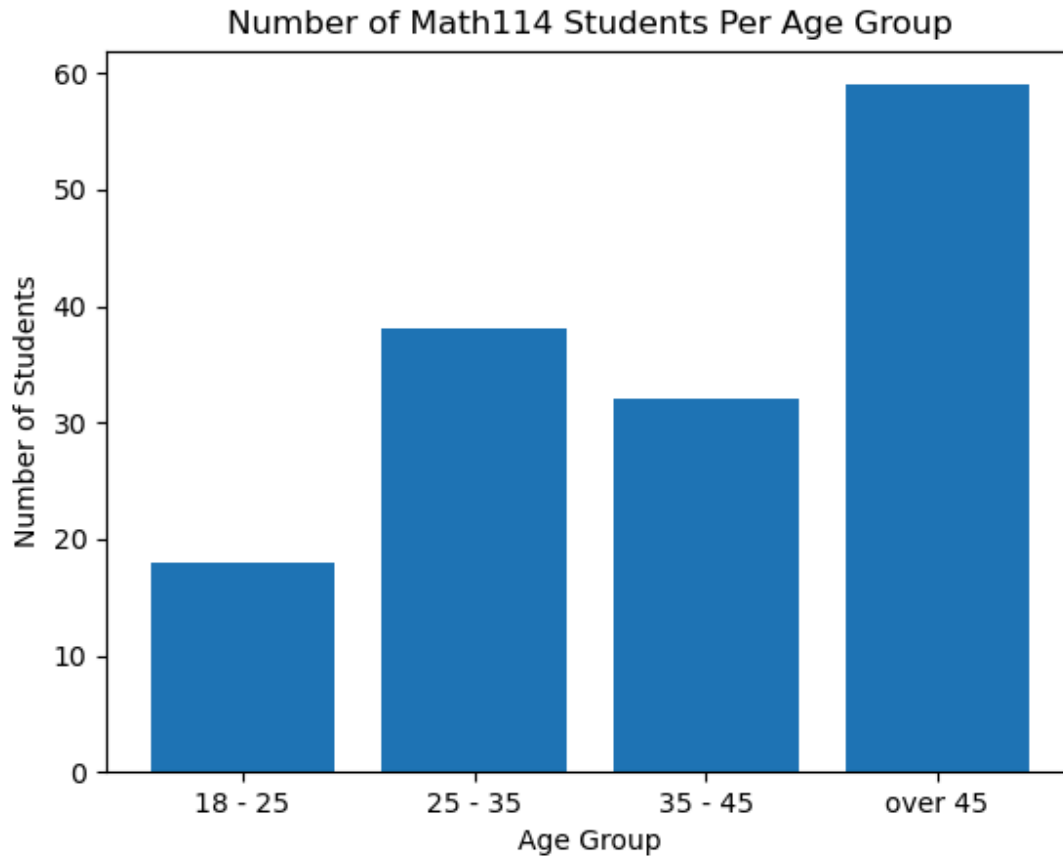
    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if ((float(y[3]))/130)*100 < 45:
                under_forty_five_math114 += 1
            elif ((float(y[3]))/130)*100 < 50:
                forty_five_forty_nine_math114 += 1
            elif ((float(y[3]))/130)*100 < 60:
                fifty_fifty_nine_math114 += 1
            elif ((float(y[3]))/130)*100 < 70:
                sixty_sixty_nine_math114 += 1
            elif ((float(y[3]))/130)*100 < 80:
                seventy_seventy_nine_math114 += 1
            elif ((float(y[3]))/130)*100 >= 80:
                over_eighty_math114 += 1

    data['Mark'] = ['under 45', '45 - 49', '50 - 59', '60 - 69', '70 - 79', 'over 80']
    data['Frequency'] = [under_forty_five_math114, forty_five_forty_nine_math114,
fifty_fifty_nine_math114, sixty_sixty_nine_math114, seventy_seventy_nine_math114,
over_eighty_math114]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "Math114 Student Marks "
    print(heading)
    print(frequency_table_df.to_string(index=False))
```

Conclusion:

The category of marks with the highest frequency is under 45 (with a frequency of 69). The frequency of the 45 - 49 group is 9. This means that over 50% of the Math 114 students received a failing grade for the exam.

Math 114 Bar Chart:



Source Code:

```
# 2.2.1 a) Use a bar chart to show the age groups (x) and numbers of Math114 students
def bar_age_group_math114():
    # count how many student in each age range and put in array
    student_age_num = np.array([])
    # counters for number of students in each age group
    eighteen_twenty_five_age_math114counter = 0
    twenty_five_thirty_five_age_math114counter = 0
    thirty_five_forty_five_age_math114counter = 0
    over_forty_five_age_math114counter = 0
    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            line_info = line.split(',')
            if 18 <= int(line_info[1]) < 25:
                eighteen_twenty_five_age_math114counter += 1
            elif 25 <= int(line_info[1]) < 35:
                twenty_five_thirty_five_age_math114counter += 1
            elif 35 <= int(line_info[1]) < 45:
                thirty_five_forty_five_age_math114counter += 1
            elif int(line_info[1]) > 45:
                over_forty_five_age_math114counter += 1
```

```

student_age_num = np.append(student_age_num, eighteen_twenty_five_age_math114counter)
student_age_num = np.append(student_age_num, twenty_five_thirty_five_age_math114counter)
student_age_num = np.append(student_age_num, thirty_five_forty_five_age_math114counter)
student_age_num = np.append(student_age_num, over_forty_five_age_math114counter)

x = np.array(['18 - 25', '25 - 35', '35 - 45', 'over 45'])
y = student_age_num

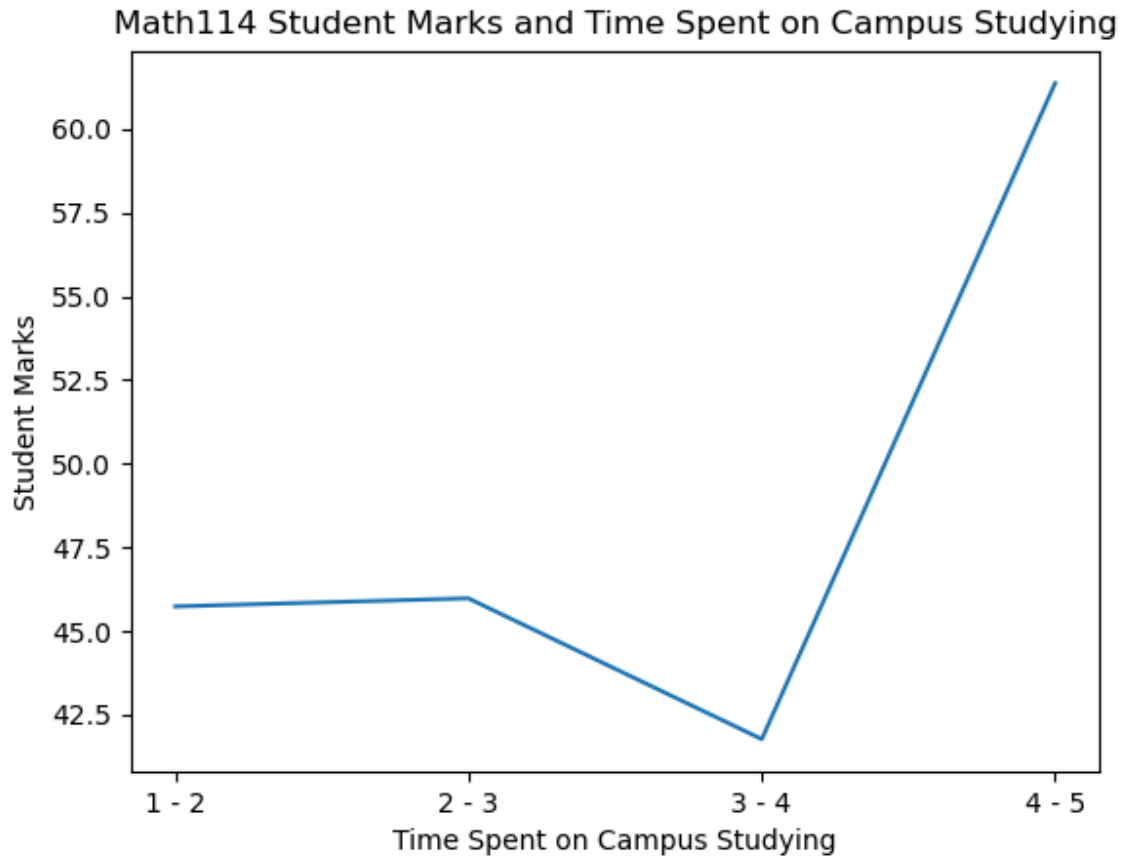
# labelling x-axis and y-axis
plt.xlabel('Age Group')
plt.ylabel('Number of Students')
# displaying the title
plt.title("Number of Math114 Students Per Age Group")
plt.bar(x, y)
plt.show()

```

Conclusion:

The age group with the lowest number of Math 114 students is the 18 - 25 group. The age group with the second lowest number of Math 114 students is the 25 - 35 group. The age group with the second highest number of Math 114 students is the 35 - 45. The age group with the highest number of Math 114 students is the over 45. This shows that there are more students over 35, than under 35.

Math 114 Line Graph:



Source Code:

```
# 2.2.2 a) Use a line graph to show if there is a correlation between higher Math114 marks(y)
and more time spent on campus studying(x).
# variables to store num of student in each time category
def line_marks_study_hrs_math114():
    num_students_one_math114 = 0
    num_students_two_math114 = 0
    num_students_three_math114 = 0
    num_students_four_math114 = 0

    sum_grades_one_math114 = 0
    sum_grades_two_math114 = 0
    sum_grades_three_math114 = 0
    sum_grades_four_math114 = 0

    # get average study hours for each group
    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if float(y[2]) < 2:
```

```

        # get percentage of each student's mark, then add it to
grades_students_one_two_math114
    sum_grades_one_math114 = sum_grades_one_math114 + (int(y[3]) / 130) * 100
    num_students_one_math114 += 1
elif float(y[2]) < 3:
    sum_grades_two_math114 = sum_grades_two_math114 + (int(y[3]) / 130) * 100
    num_students_two_math114 += 1
elif float(y[2]) < 4:
    sum_grades_three_math114 = sum_grades_three_math114 + (int(y[3]) / 130) * 100
    num_students_three_math114 += 1
elif float(y[2]) <= 5:
    sum_grades_four_math114 = sum_grades_four_math114 + (int(y[3]) / 130) * 100
    num_students_four_math114 += 1

avg_one_math114 = sum_grades_one_math114 / num_students_one_math114
avg_two_math114 = sum_grades_two_math114 / num_students_two_math114
avg_three_math114 = sum_grades_three_math114 / num_students_three_math114
avg_four_math114 = sum_grades_four_math114 / num_students_four_math114

x = np.array(['1 - 2', '2 - 3', '3 - 4', '4 - 5'])
y = np.array([avg_one_math114, avg_two_math114, avg_three_math114, avg_four_math114])

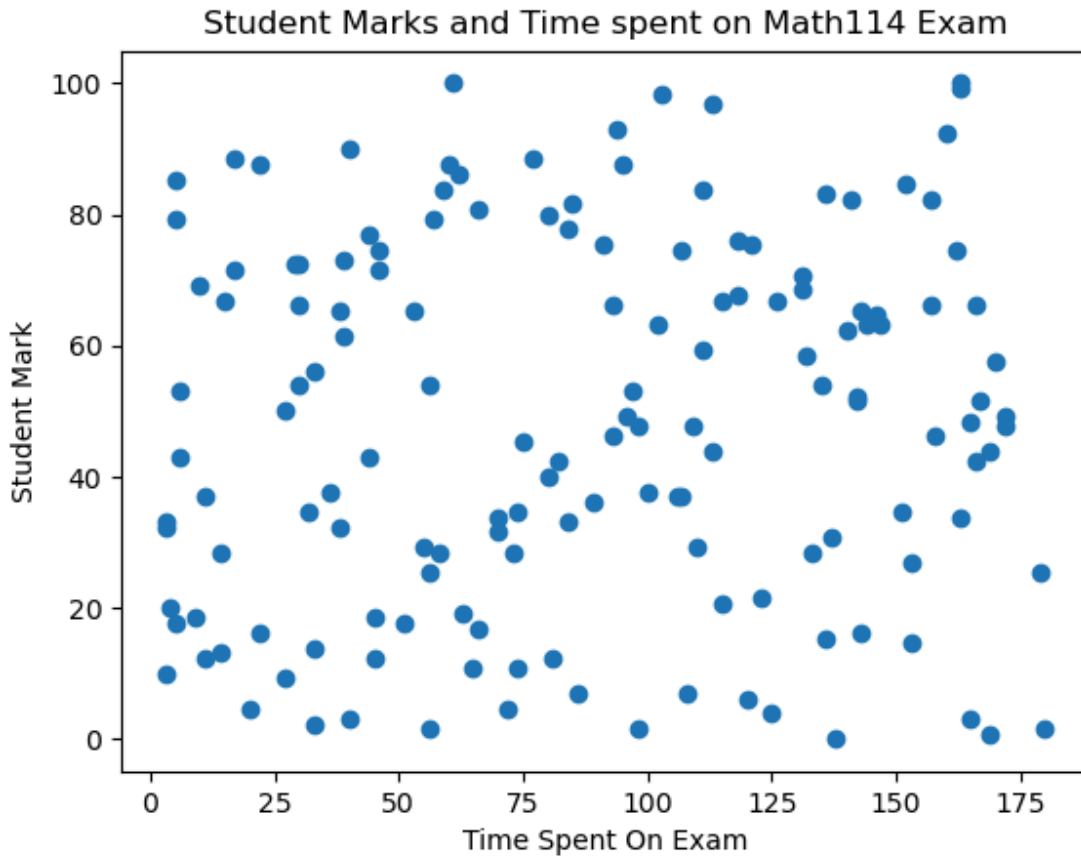
plt.plot(x, y)
# labelling x-axis and y-axis
plt.xlabel('Time Spent on Campus Studying')
plt.ylabel('Student Marks')
# displaying the title
plt.title('Math114 Student Marks and Time Spent on Campus Studying')
plt.show()

```

Conclusion:

The group of students who spent an average of 4 - 5 hours on campus got the highest collective exam mark average compared to other groups.

Math 114 Scatter Plots:



Source Code:

```
# sort array by time taken on exam
def sort_array(array):
    # bubble sort
    for first_idx, first_item in enumerate(array):
        for second_idx, second_item in enumerate(array):
            if int(array[first_idx][4]) < int(array[second_idx][4]):
                # TODO: make sort make sense
                smaller = array[second_idx]
                bigger = array[first_idx]
                array[first_idx] = smaller
                array[second_idx] = bigger
    return array

def scatter_marks_time_math114():
    x = np.array([]) # time taken
    y = np.array([]) # student mark
    new_array = []
    with open('student_data_math114.csv', mode='r', newline='') as file:
        file.readline()
        for line in file:
            stripped_line = line.rstrip('\r\n')
            line_items = stripped_line.split(',')
            new_array.append(line_items)
```

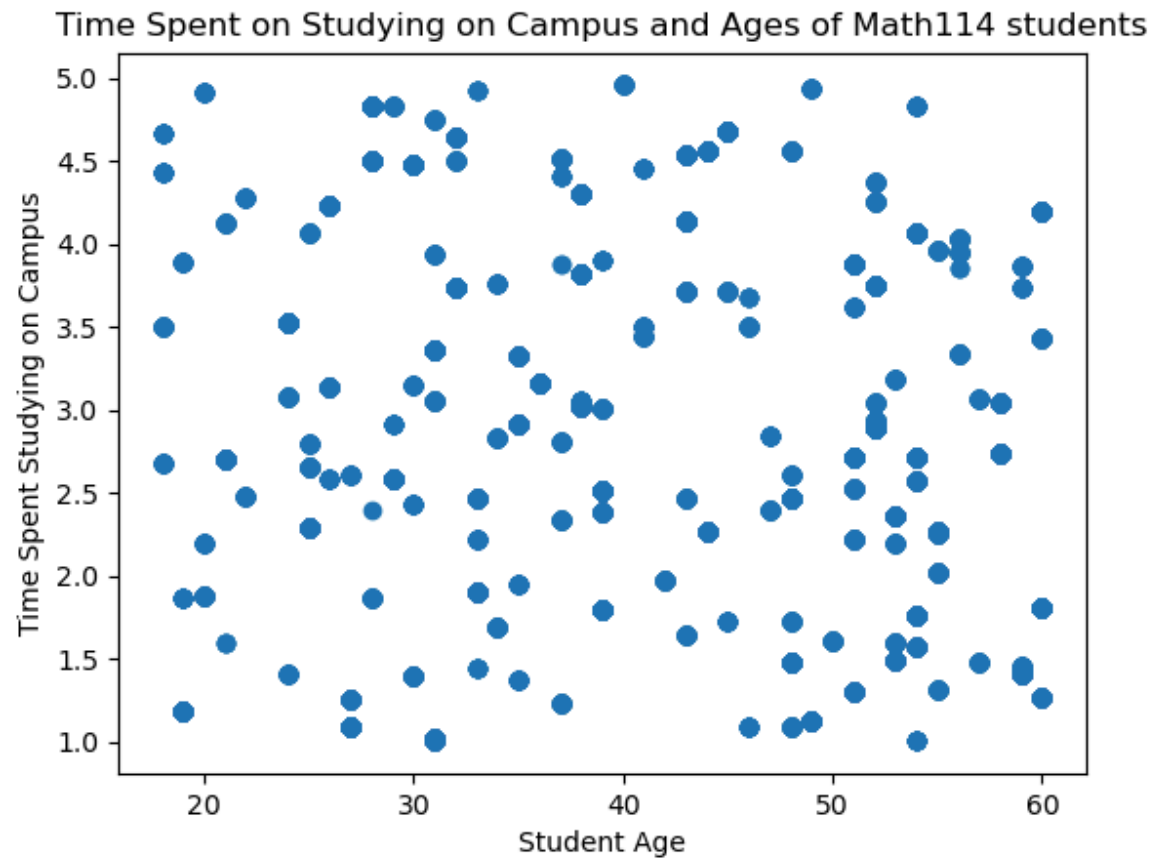
```
new_array.append(line_items)

# sort array of lines in ascending order by time which is index 4 of each line
sorted_array = sort_array(new_array)
for sorted_item in sorted_array:
    # [student_number, age_group, hours_studying, student_mark, time_taken_on_exam]
    x = np.append(x, int(sorted_item[4]))
    y = np.append(y, (float(sorted_item[3])/130) * 100)

plt.scatter(x, y)
# labelling x-axis and y-axis
plt.xlabel('Time Spent On Exam')
plt.ylabel('Student Mark')
# # displaying the title
plt.title('Student Marks and Time spent on Math114 Exam')
plt.show()
```

Conclusion:

There is no correlation between exam marks and time spent on exam.



Source Code:

```
#2.2.4 a) Use a scatter chart to plot the relationship between time spent on campus(y)and age
of Math114 students.
# sort by hours spent on campus studying
def sort_array_by_study_time(array):
    # bubble sort
    for first_idx, first_item in enumerate(array):
        for second_idx, second_item in enumerate(array):
            if float(array[first_idx][2]) < float(array[second_idx][2]):
                # TODO: make sort make sense
                smaller = array[second_idx]
                bigger = array[first_idx]
                array[first_idx] = smaller
                array[second_idx] = bigger
    return array

def scatter_study_time_age_math114():
    x = np.array([])
    y = np.array([])
    new_array = []
    with open('student_data_math114.csv', mode='r', newline='') as f:
        f.readline()
        for line in f:
            stripped_line = line.rstrip('\r\n')
            line_items = stripped_line.split(',')
            new_array.append(line_items)
```

```
        # sort array of lines in ascending order by time which is index 2 of each line
        sorted_array = sort_array_by_study_time(new_array)
        for sorted_item in sorted_array:
            # [student_number, age_group, hours_studying, student_mark,
time_taken_on_exam]
            x = np.append(x, float(sorted_item[1]))
            y = np.append(y, float(sorted_item[2]))

    plt.scatter(x, y)
    #labelling x-axis and y-axis
    plt.xlabel('Student Age')
    plt.ylabel('Time Spent Studying on Campus')
    # displaying the title
    plt.title('Time Spent on Studying on Campus and Ages of Math114 students')
    plt.show()
```

Conclusion:

There is no correlation between a student's age and the time they spent studying on campus.

CS 114 Data Visualisations

CS 114 Frequency Tables:

Table:

Average Hours Spent on Campus by CS114 Students	
Average Hours Studying	Frequency
1 - 2	38
2 - 3	36
3 - 4	38
4 - 5	38

Source Code:

```
# 2.1.1 b) Create frequency table showing average hours CS114 students spent studying on campus
def freq_avg_study_hrs_cs114():
    data = {}
    # count number of students in each category of hours using the following counters
    one_two_hours_cs114counter = 0
    two_three_hours_cs114counter = 0
    three_four_hours_cs114counter = 0
    four_five_hours_cs114counter = 0
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if float(y[2]) < 2:
                one_two_hours_cs114counter += 1
            elif float(y[2]) < 3:
                two_three_hours_cs114counter += 1
            elif float(y[2]) < 4:
                three_four_hours_cs114counter += 1
            elif float(y[2]) <= 5:
                four_five_hours_cs114counter += 1

    data['Average Hours Studying'] = ['1 - 2', '2 - 3', '3 - 4', '4 - 5']
    data['Frequency'] = [one_two_hours_cs114counter, two_three_hours_cs114counter,
three_four_hours_cs114counter, four_five_hours_cs114counter]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "Average Hours Spent on Campus by CS114 Students"
    print(heading)
    print(frequency_table_df.to_string(index=False))
```

Conclusion:

The number of students in each group of Average Study Hours is evenly distributed.

Table:

CS114 Students Age	
Age Group	Frequency
18 - 25	27
25 - 35	40
35 - 45	29
over 45	54

Source Code:

```
# 2.1.2 b) Create frequency table showing student ages of CS114 Students
def freq_student_ages_cs114():
    data = {}
    eighteen_twenty_five_age_cs114counter = 0
    twenty_five_thirty_five_age_cs114counter = 0
    thirty_five_forty_five_age_cs114counter = 0
    over_forty_five_age_cs114counter = 0
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if int(y[1]) <= 25:
                eighteen_twenty_five_age_cs114counter += 1
            elif int(y[1]) <= 35:
                twenty_five_thirty_five_age_cs114counter += 1
            elif int(y[1]) <= 45:
                thirty_five_forty_five_age_cs114counter += 1
            elif int(y[1]) > 45:
                over_forty_five_age_cs114counter += 1

    data['Age Group'] = ['18 - 25', '25 - 35', '35 - 45', 'over 45']
    data['Frequency'] = [eighteen_twenty_five_age_cs114counter,
twenty_five_thirty_five_age_cs114counter, thirty_five_forty_five_age_cs114counter,
over_forty_five_age_cs114counter]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "CS114 Students Age "
    print(heading)
    print(frequency_table_df.to_string(index=False))
```

Conclusion:

The age group with the lowest number of Math 114 students is the 18 - 25 group. The age group with the second lowest number of Math 114 students is the 35 - 45. The age group with the second highest number of Math 114 students is the 25 - 35 group. The age group with the highest number of Math 114 students is the over 45. This shows that there are more students over 35, than under 35.

Table:

CS114 Student Marks	
Mark	Frequency
under 45	60
45 - 49	11
50 - 59	19
60 - 69	21
70 - 79	10
over 80	29

Source Code:

```
# 2.1.3 b) Create frequency table showing Student marks of CS114 Students
def freq_student_marks_cs114():
    data = {}
    under_forty_five_cs114 = 0
    forty_five_forty_nine_cs114 = 0
    fifty_fifty_nine_cs114 = 0
    sixty_sixty_nine_cs114 = 0
    seventy_seventy_nine_cs114 = 0
    over_eighty_cs114 = 0

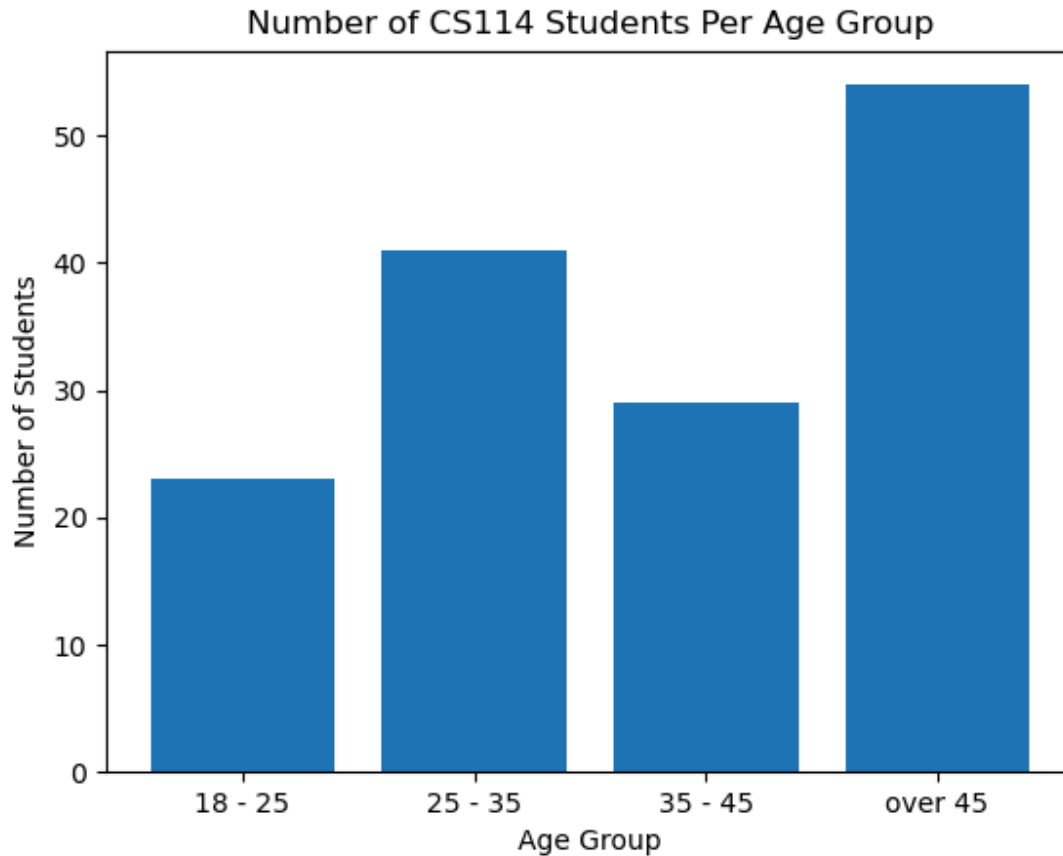
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if ((float(y[3]))/130)*100 < 45:
                under_forty_five_cs114 += 1
            elif ((float(y[3]))/130)*100 < 50:
                forty_five_forty_nine_cs114 += 1
            elif ((float(y[3]))/130)*100 < 60:
                fifty_fifty_nine_cs114 += 1
            elif ((float(y[3]))/130)*100 < 70:
                sixty_sixty_nine_cs114 += 1
            elif ((float(y[3]))/130)*100 < 80:
                seventy_seventy_nine_cs114 += 1
            elif ((float(y[3]))/130)*100 >= 80:
                over_eighty_cs114 += 1

    data['Mark'] = ['under 45','45 - 49','50 - 59','60 - 69','70 - 79','over 80']
    data['Frequency'] = [under_forty_five_cs114, forty_five_forty_nine_cs114,
fifty_fifty_nine_cs114, sixty_sixty_nine_cs114, seventy_seventy_nine_cs114, over_eighty_cs114]
    # Creating a DataFrame from the frequency table data
    frequency_table_df = pd.DataFrame(data)
    heading = "CS114 Student Marks "
    print(heading)
    print(frequency_table_df.to_string(index=False))
```

Conclusion:

More than 50% of the CS 14 student received a passing grade.

CS 114 Bar Chart:



Source Code:

```
# 2.2.1 b) Use a bar chart to show the age groups (x) and numbers of CS114 students
def bar_age_group_cs114():
    # count how many student in each age range and put in array
    student_age_num = np.array([])
    # counters for number of students in each age group
    eighteen_twenty_five_age_cs114counter = 0
    twenty_five_thirty_five_age_cs114counter = 0
    thirty_five_forty_five_age_cs114counter = 0
    over_forty_five_age_cs114counter = 0
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            line_info = line.split(',')
            if 18 <= int(line_info[1]) < 25:
                eighteen_twenty_five_age_cs114counter += 1
            elif 25 <= int(line_info[1]) < 35:
                twenty_five_thirty_five_age_cs114counter += 1
            elif 35 <= int(line_info[1]) < 45:
                thirty_five_forty_five_age_cs114counter += 1
            elif int(line_info[1]) > 45:
                over_forty_five_age_cs114counter += 1
```

```

student_age_num = np.append(student_age_num, eighteen_twenty_five_age_cs114counter)
student_age_num = np.append(student_age_num, twenty_five_thirty_five_age_cs114counter)
student_age_num = np.append(student_age_num, thirty_five_forty_five_age_cs114counter)
student_age_num = np.append(student_age_num, over_forty_five_age_cs114counter)

x = np.array(['18 - 25', '25 - 35', '35 - 45', 'over 45'])
y = student_age_num

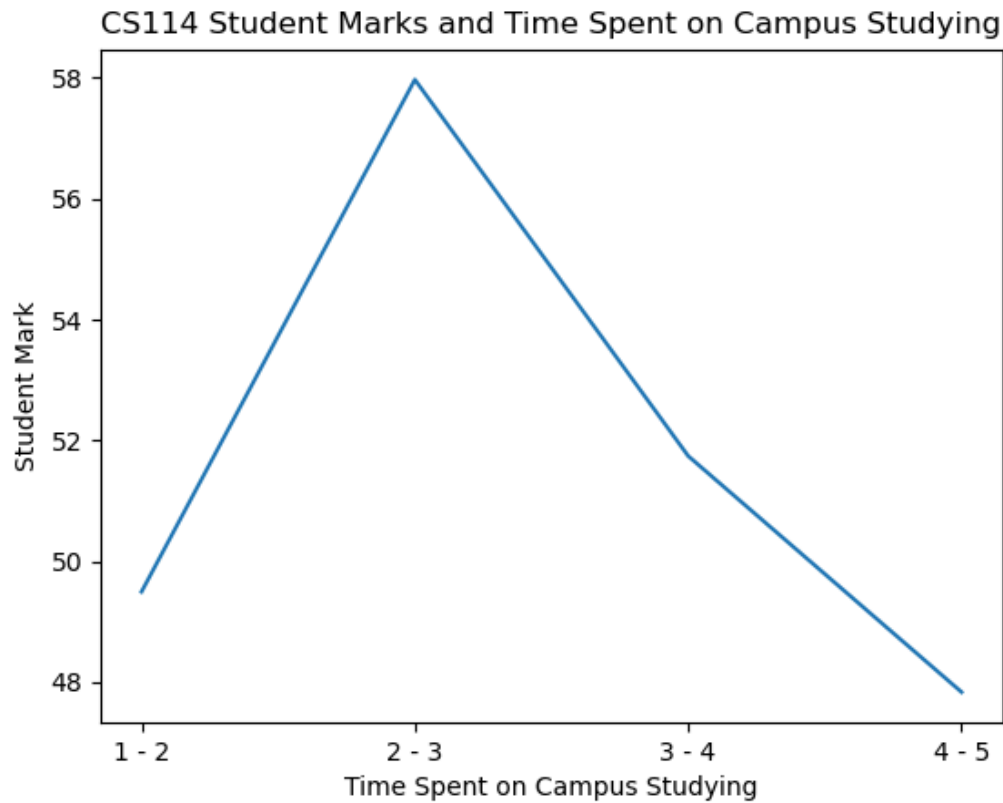
# labelling x-axis and y-axis
plt.xlabel('Age Group')
plt.ylabel('Number of Students')
# displaying the title
plt.title("Number of CS114 Students Per Age Group")
plt.bar(x, y)
plt.show()

```

Conclusion:

The age group with the lowest number of Math 114 students is the 18 - 25 group. The age group with the second lowest number of Math 114 students is the 35 - 45. The age group with the second highest number of Math 114 students is the 25 - 35 group. The age group with the highest number of Math 114 students is the over 45. This shows that there are more students over 35, than under 35.

CS 114 Line Graph:



Source Code:

```
#2.2.2 b) Use a line graph to show if there is a correlation between higher CS114 marks(y) and
more time spent on campus(x).
def line_marks_study_hrs_cs114():
    num_students_one_cs114 = 0
    num_students_two_cs114 = 0
    num_students_three_cs114 = 0
    num_students_four_cs114 = 0

    sum_grades_one_cs114 = 0
    sum_grades_two_cs114 = 0
    sum_grades_three_cs114 = 0
    sum_grades_four_cs114 = 0

    # get average study hours for each group
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for i in range(150):
            line = f.readline()
            y = line.split(',')
            if float(y[2]) < 2:
                # get percentage of each student's mark, then add it to
                grades_students_one_two_math114
                sum_grades_one_cs114 = sum_grades_one_cs114 + (float(y[3]) / 130) * 100
                num_students_one_cs114 += 1
```

```

        elif float(y[2]) < 3:
            sum_grades_two_cs114 = sum_grades_two_cs114 + (float(y[3]) / 130) * 100
            num_students_two_cs114 += 1
        elif float(y[2]) < 4:
            sum_grades_three_cs114 = sum_grades_three_cs114 + (float(y[3]) / 130) * 100
            num_students_three_cs114 += 1
        elif float(y[2]) < 5:
            sum_grades_four_cs114 = sum_grades_four_cs114 + (float(y[3]) / 130) * 100
            num_students_four_cs114 += 1

# get percentage of each student's mark, then add it to grades_students_one_two_math114
avg_one_cs114 = sum_grades_one_cs114 / num_students_one_cs114
avg_two_cs114 = sum_grades_two_cs114 / num_students_two_cs114
avg_three_cs114 = sum_grades_three_cs114 / num_students_three_cs114
avg_four_cs114 = sum_grades_four_cs114 / num_students_four_cs114

x = np.array(['1 - 2', '2 - 3', '3 - 4', '4 - 5'])
y = np.array([avg_one_cs114, avg_two_cs114, avg_three_cs114, avg_four_cs114])

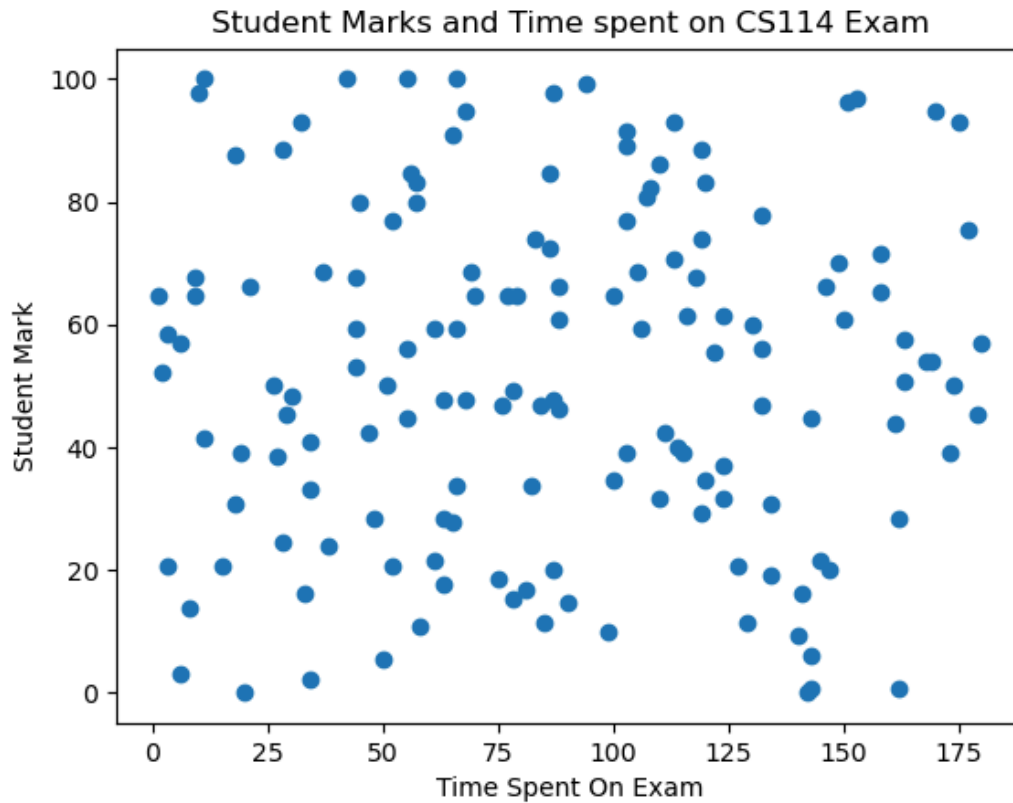
plt.plot(x, y)
# labelling x-axis and y-axis
plt.xlabel('Time Spent on Campus Studying')
plt.ylabel('Student Mark')
# displaying the title
plt.title('CS114 Student Marks and Time Spent on Campus Studying')
plt.show()

```

Conclusion:

The group of students who spent an average of 2 - 3 hours on campus got the highest collective exam mark average compared to other groups.

CS 114 Scatter Plots:



Source Code:

```
def sort_array(array): # sort by time taken
    # bubble sort
    for first_idx, first_item in enumerate(array):
        for second_idx, second_item in enumerate(array):
            if int(array[first_idx][4]) < int(array[second_idx][4]):
                # TODO: make sort make sense
                smaller = array[second_idx]
                bigger = array[first_idx]
                array[first_idx] = smaller
                array[second_idx] = bigger
    return array

# 2.2.3 b) Use a scatter chart to plot each student's mark(y) and the time(x) taken on the
CS114 examination, in minutes.
def scatter_marks_time_cs114():
    x = np.array([]) # time taken
    y = np.array([]) # student mark
    new_array = []
    with open('student_data_cs114.csv', mode='r', newline='') as file:
        file.readline()
        for line in file:
            stripped_line = line.rstrip('\r\n')
            line_items = stripped_line.split(',')
            new_array.append(line_items)
```

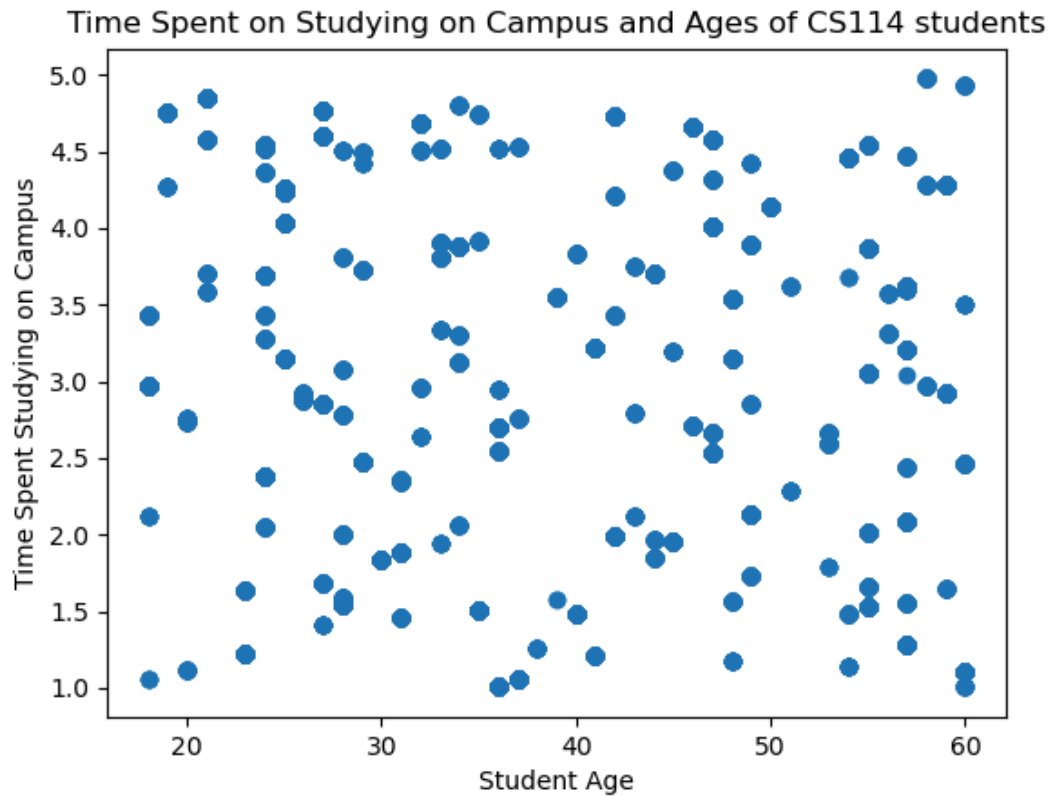


```
# sort array of lines in ascending order by time which is index 4 of each line
sorted_array = sort_array(new_array)
for sorted_item in sorted_array:
    # [student_number, age_group, hours_studying, student_mark, time_taken_on_exam]
    x = np.append(x, int(sorted_item[4]))
    y = np.append(y, (float(sorted_item[3]) / 130) * 100)

plt.scatter(x, y)
# labelling x-axis and y-axis
plt.xlabel('Time Spent On Exam')
plt.ylabel('Student Mark')
# # displaying the title
plt.title('Student Marks and Time spent on CS114 Exam')
plt.show()
```

Conclusion:

There is no correlation between exam marks and time spent on exam.



Source Code:

```
# 2.2.4 b) Use a scatter chart to plot the relationship between time spent on campus(y)and
age of CS114 students.
rray[second_idx] = bigger
    return array
# sort by hours spent on campus studying
def sort_array_by_study_time(array):
    # bubble sort
    for first_idx, first_item in enumerate(array):
        for second_idx, second_item in enumerate(array):
            if float(array[first_idx][2]) < float(array[second_idx][2]):
                # TODO: make sort make sense
                smaller = array[second_idx]
                bigger = array[first_idx]
                array[first_idx] = smaller
                array[second_idx] = bigger
    return array

def scatter_study_time_age_cs114():
    x = np.array([]) # time spent
    y = np.array([])
    new_array = []
    with open('student_data_cs114.csv', mode='r', newline='') as f:
        f.readline()
        for line in f:
            stripped_line = line.rstrip('\r\n')
            line_items = stripped_line.split(',')
            new_array.append(line_items)
```

```

        # sort array of lines in ascending order by time which is index 2 of each line
        sorted_array = sort_array_by_study_time(new_array)
        for sorted_item in sorted_array:
            # [student_number, age_group, hours_studying, student_mark,
time_taken_on_exam]
            x = np.append(x, float(sorted_item[1]))
            y = np.append(y, float(sorted_item[2]))

plt.scatter(x, y)
#labeling x-axis and y-axis
plt.xlabel('Student Age')
plt.ylabel('Time Spent Studying on Campus')
# displaying the title
plt.title('Time Spent on Studying on Campus and Ages of CS114 students')
plt.show()

```

Conclusion:

There is no correlation between a student's age and the time they spent studying on campus.